Scalable RVV Design and Architecture
from 8-bit to 64-bit

BTRON BX Series
Independent Systems Architecture Study

November 2026

Contents

Contents i

Preface v

A Note to the RTL Reader vii

Notation and Conventions ix

1 Machines That Remain MSX and Amiga 1
1.1 The place of the original machines . . . 1
1.2 What compatible means in RTL . . . 1
1.3 Why the peripherals survive the change of processor . . . 2
1.4 The four-machine contract . . . 2

2 The RISC-V Ladder, from an 8-bit Bus to a 64-bit Hart 3
2.1 Datapath width is not XLEN . . . 3
2.2 One decode for scalar and vector . . . 3
2.3 Latency you should publish . . . 4
2.4 What moves upward unchanged . . . 4

3 SVM Profiles and the Ratified Vector Extensions 5
3.1 The names . . . 5
3.2 Why SVM-8 is not Zve32x . . . 5
3.3 The `misa` rule . . . 6

3.4 Xsvm . . . 6
3.5 Fixed point without a private lane type . . . 6

4 Vector State **7**
4.1 The register file . . . 7
4.2 Configuration registers . . . 7
4.3 Legality at configuration time . . . 8
4.4 Tail and mask policy . . . 8
4.5 Precise traps . . . 8

5 The Instruction Classes **9**
5.1 A minimum that is still RVV . . . 9
5.2 Configuration . . . 9
5.3 Loads and stores . . . 9
5.4 Integer arithmetic . . . 9
5.5 Reductions, masks, and permutes . . . 10
5.6 Floating point . . . 10
5.7 Custom geometry . . . 10
5.8 Instructions you will be tempted to skip . . . 10

6 Lanes, Register Files, and Memory on an FPGA **11**
6.1 How many lanes . . . 11
6.2 The register file . . . 11
6.3 Chaining . . . 12
6.4 Memory width . . . 12
6.5 What to read in other people's cores . . . 12

7 BX-8: An MSX-Compatible Machine with SVM-8 **13**
7.1 The board-level picture . . . 13
7.2 The hart . . . 13
7.3 SVM-8 . . . 13
7.4 Opcode map for SVM-8 . . . 14
7.4.1 Configuration — `vsetvli` . . . 14
7.4.2 Unit-stride load — `vle8.v` . . . 14
7.4.3 Integer arithmetic (OPIVV / OPIVX) . . . 15
7.4.4 Compare (mask result) . . . 15
7.4.5 Move and splat . . . 16
7.4.6 Mask logicals . . . 16
7.5 Assembler kernels that must run . . . 16
7.5.1 fill8 — fill a region with a byte . . . 16
7.5.2 copy8 — unit-stride byte copy . . . 16
7.5.3 mask8 — selective fill under a generated mask . . . 17
7.5.4 add8 — element-wise byte add with saturation . . . 17
7.6 What you elaborate . . . 17

8 BX-16: An Amiga-Compatible Machine with SVM-16 **19**
8.1 The board-level picture . . . 19
8.2 SVM-16 . . . 19
8.3 Widening, not a private fixed-point format . . . 19
8.4 Xsvm . . . 20
8.5 Opcode map for SVM-16 . . . 20

8.5.1 Configuration . . . 20
8.5.2 Unit-stride load/store (EEW = 8 and 16) . . . 20
8.5.3 Constant-stride load — `vlse16.v` . . . 20
8.5.4 Integer arithmetic (SEW = 8, 16) . . . 21
8.5.5 Widening multiply and narrowing . . . 21
8.5.6 Reduction and slide . . . 21
8.5.7 Optional Xsvm (custom-0) . . . 21
8.6 Assembler kernels that must run . . . 22
8.6.1 sew16 — halfword fill . . . 22
8.6.2 vwmul16 — widening multiply then narrow with saturation . . . 22
8.6.3 stride16 — constant-stride gather of halfwords . . . 23
8.6.4 redsum16 — integer reduction . . . 23
8.7 One hart . . . 23
8.8 Elaboration . . . 23

9 BX-32: Embedded Floating Point with Zve32f 25
9.1 The first ratified claim . . . 25
9.2 Floating point in the lane . . . 25
9.3 Where SVM-16 software feels the change . . . 25
9.4 Opcode map for Zve32f (BX-32) . . . 26
9.4.1 Configuration . . . 26
9.4.2 Unit-stride, stride and indexed load/store . . . 26
9.4.3 Integer arithmetic (SEW = 8, 16, 32) . . . 26
9.4.4 Floating-point arithmetic (SEW = 32) . . . 26
9.4.5 Mask and move . . . 27
9.5 Assembler kernels that must run . . . 27
9.5.1 sew32 — word fill . . . 27
9.5.2 fma32 — fused multiply-add strip . . . 28
9.5.3 index32 — indexed load (gather) . . . 28
9.6 Two harts . . . 28

10 BX-64: Application Vectors with V and Zve64d 29
10.1 The top of the ladder . . . 29
10.2 What arrives with sixty-four-bit elements . . . 29
10.3 Opcode map for Zve64d / V (BX-64) . . . 30
10.3.1 Configuration . . . 30
10.3.2 Unit-stride load at SEW=64 — `vle64.v` . . . 30
10.3.3 Integer / FP arithmetic (OP-V) at any SEW including 64 . . . 30
10.3.4 Mask and move . . . 31
10.4 Assembler kernels that must run . . . 31
10.4.1 sew64 — double-word fill . . . 31
10.4.2 fma64 — double-precision fused multiply-add . . . 31
10.4.3 index64 — indexed load at SEW=64 . . . 31
10.5 Two harts, local memory, DMA . . . 32
10.6 Xsvm on this machine . . . 32

11 One Bluespec Description, Four Profiles 33
11.1 Complete opcode contract . . . 33
11.2 Aggregated opcode tables (ISA) . . . 34
11.2.1 Canonical layouts (32-bit, two equal-width half-words) . . . 34
11.2.2 Mnemonic coverage by profile . . . 35

11.2.3 Explicit summary — SVM-8 (BX-8) . . . 35
11.2.4 Explicit summary — SVM-16 (BX-16) . . . 35
11.2.5 Explicit summary — Zve32f (BX-32) . . . 35
11.2.6 Explicit summary — Zve64d / V (BX-64) . . . 36
11.2.7 Key constants (funct6 / width / mop) . . . 36
11.3 Parameters . . . 36
11.4 Interfaces . . . 37
11.5 The configuration rule . . . 37
11.6 Arithmetic rules . . . 37
11.7 What SVM-8 still contains . . . 37

12 Bring-Up, from `vsetvl` to a Running Kernel **39**
12.1 Order . . . 39
12.2 The first vector add . . . 39
12.3 Bus contention on the compatible machines . . . 40
12.4 When to leave the simulator . . . 40

13 Tests That Travel Up the Ladder **41**
13.1 One suite, four expectations . . . 41
13.2 The fill . . . 41
13.3 Illegal SEW . . . 41
13.4 Xsvm tests are separate . . . 42
13.5 Random parcels . . . 42

14 What Fits on the First FPGA **43**
14.1 The file, in bytes . . . 43
14.2 DSP blocks . . . 43
14.3 The first images . . . 43
14.4 Clocks . . . 44
14.5 A finished first board . . . 44

Profile Card **45**

Checklist against RVV 1.0 **47**

MSX and Amiga Compatibility Checklist **49**
BX-8, in place of an MSX . . . 49
BX-16, in place of an Amiga . . . 49
BX-32 and BX-64 . . . 49

Exercises **51**

Glossary **53**

References **55**

Preface

This book is a construction manual for one family of FPGA machines. The family is called BX. Each member is a RISC-V hart whose data path grows from eight bits to sixty-four, and whose vector unit grows with it under the RISC-V Vector Extension.

The eight-bit member is built so that it can take the place of an MSX: the slots, the video display processor, the sound generator, and the memory map stay recognisable. The sixteen-bit member is built so that it can take the place of an Amiga: a sixteen-bit bus, chipset-style shared memory, and the traffic patterns of a blitter and a copper. In both cases the processor that issues the instructions is RISC-V. The original machines remain the reference for peripherals and for the work the software has to do. The hart is the part that stands in their place.

Above those two machines the book stops speaking of profiles and starts speaking of ratified extensions. BX-32 implements Zve32f. BX-64 implements the application extension V, or at least Zve64d. A program written against the narrow profiles is still an RVV program. It uses the same registers, the same configuration instruction, and the same opcode map. What changes is the widest element the machine accepts.

The text is for a reader who will write the register transfer logic, preferably in Bluespec SystemVerilog, and who will synthesise it. It is not a programmer's companion to the vector intrinsics, and it is not a history of the home computers whose peripheral sets the machines keep.

The architectural choices that the rest of the book treats as fixed are these. The integer register width, XLEN, is 32 on BX-8, BX-16, and BX-32, and 64 on BX-64. There is no separate eight-bit or sixteen-bit base instruction set; the narrowness is in the ALU, the bus, and the vector element length. The vector file always has thirty-two registers. Geometry operations that the vector specification does not define live in an optional custom extension and are silent unless a board asks for them.

A Note to the RTL Reader

Read the book in order the first time. The profiles in Part I are short on purpose. They exist so that later chapters can say “SVM-8” instead of repeating a list of legal element widths.

Then build in the order of Part IV. A scalar RV32I that fetches, decodes, and writes back is the whole of the first milestone. The vector unit begins as a configuration instruction and a single vector add. Memory, widening, masks, and floating point come after that, and only on the machines whose profile includes them.

Keep three files open while you read: the RVV specification, the profile card at the back of this book, and your elaboration parameters. If a paragraph and the profile card disagree, the card is the contract for the implementation and the paragraph is wrong.

Two habits will save you more time than any micro-architectural trick. First, make an illegal configuration raise an illegal-instruction exception in the same cycle you decode `vsetvl`, not deep in a lane. Second, do not advertise a ratified extension string until every instruction that string requires is present and tested. SVM-8 and SVM-16 are profiles of the encoding. They are not Zve32x.

Simulation comes before the board. A strip-mined fill that runs, with different vector lengths, on all four elaborated designs is the acceptance test of the ladder. Place and route is the acceptance test of the budget chapter, not of the architecture.

Notation and Conventions

Table 1: Names used throughout the book.

Name	Meaning
XLEN	Width of an integer register, in bits. 32 or 64.
Datapath	Width of the integer ALU and of the data bus, in bits.
ELEN	Widest vector element the implementation supports.
VLEN	Bits in one vector register. A power of two, at least ELEN.
SEW	Selected element width for the current configuration.
EEW	Effective element width of one instruction.
VL	Number of elements the current instruction processes.
VLMAX	Maximum VL for the current SEW and LMUL.
LMUL	Register grouping factor.
SVM-8	BX-8 profile: ELEN 8, VLEN 32, EEW 8.
SVM-16	BX-16 profile: ELEN 16, VLEN 128, EEW 8 and 16.

Instructions are written in the usual assembly form, `vadd.vv v1, v2, v3`. Configuration is `vsetvl` or `vsetvli`. The thirty-two vector registers are `v0` through `v31`. The mask register is `v0`.

A *profile* is a named subset of RVV behaviour that this book specifies and that the specification does not ratify under that name. A *ratified extension* is a string the RISC-V manuals define: Zve32f, Zve64d, or V.

Bluespec fragments are illustrative. They show interfaces and rules, not a vendor core. Module names use the prefix `Bx`.

The words MSX and Amiga, in this book, name a peripheral and bus contract. They do not name the original microprocessor. BX-8 is MSX-compatible in that contract and can replace an MSX in a rack or on a desk. BX-16 is Amiga-compatible in that contract and can replace an Amiga in the same sense.

Chapter 1

Machines That Remain MSX and Amiga

1.1 The place of the original machines

The MSX and the Amiga solved real-time picture and sound with small memories, narrow buses, and specialised peripheral chips. Those chips, and the way software talks to them, are the part worth keeping in register-transfer form. A student who can read a video display processor written as rules will understand a later display controller. A student who can read a shared sixteen-bit bus with several masters will understand a later DMA fabric.

BX-8 is the MSX-compatible member. Its slots, video display processor, programmable sound generator, keyboard matrix, and cartridge signalling follow the MSX map closely enough that a board can sit where an MSX sat: same kind of picture, same kind of sound, same habit of I/O ports. The processor is an RV32I hart with an eight-bit data path and the SVM-8 vector profile.

BX-16 is the Amiga-compatible member. Its data bus is sixteen bits wide. Memory is shared among the processor, a blitter-class mover, a video fetch, and an audio path, in the spirit of the Amiga chipset. The processor is an RV32I hart with a sixteen-bit data path and the SVM-16 vector profile. It can take the place of an Amiga in the same practical sense: a single FPGA image that runs a sixteen-bit desktop-class peripheral set and a vector unit sized for blits, mixing, and small transforms.

1.2 What compatible means in RTL

Compatibility, here, is a list you can tick on a board, not a promise that every original binary image will execute.

- Address ranges and register layouts of the kept peripherals match the reference machines within documented differences.
- Bus width matches: eight bits on BX-8, sixteen bits on BX-16.
- Cycle behaviour of the peripherals is specified and tested against the reference documents you choose to follow.

- Software that is written for the BX hart, in RV32I and in the vector profile, drives those peripherals.
- A thin compatibility layer, if you write one, translates or interprets guest code. It is software on the hart, not a second integer pipeline hidden in the decode stage.

The original processors remain the reference for what those machines could do with a given number of cycles. They are not instantiated beside the hart. One fetch path, one decode, one writeback.

1.3 Why the peripherals survive the change of processor

A video display processor does not become simpler because the CPU is RISC-V. Sprites, tiles, blanking, and a shared memory port are the same problem on an MSX-compatible board as they were on the original. Writing them in Bluespec forces the arbitration into rules that the compiler schedules. That is the infrastructure BX-32 and BX-64 reuse when the picture is no longer a tilemap and the bus is no longer eight bits wide.

The same is true of the sixteen-bit machine. A blitter is a programmed DMA with a small arithmetic step. SVM-16 can perform the arithmetic in the vector unit, or the blitter can remain a peripheral master. Both organisations are legal on BX-16. The book recommends keeping the peripheral master for compatibility of timing, and using the vector unit for new software.

1.4 The four-machine contract

Table 1.1: What each machine is for.

Machine	Can replace	Vector contract
BX-8	an MSX	SVM-8, integer, EEW 8
BX-16	an Amiga	SVM-16, integer, EEW 8 and 16
BX-32	—	Zve32f, with FP32
BX-64	—	V or Zve64d, with FP64

BX-32 and BX-64 do not pretend to be a Saturn, a Dreamcast, a PlayStation 2, or a Nintendo 64. They keep the organisational lessons of those machines: more than one hart, local memory, and a vector unit wide enough for geometry. Their compatibility target is the RISC-V specification, not a cartridge port.

Chapter 2

The RISC-V Ladder, from an 8-bit Bus to a 64-bit Hart

2.1 Datapath width is not XLEN

RISC-V defines integer registers of thirty-two or sixty-four bits. It does not define a base with eight-bit or sixteen-bit registers. BX-8 and BX-16 honour that. XLEN is 32. The instructions in memory are the ordinary thirty-two-bit RV32I encodings. The ALU and the data bus are narrower than XLEN, so a word operation takes several cycles.

On BX-8 a load word is four bus beats, a load halfword is two, and a load byte is one. Shifts and adds iterate over bytes, low byte first, with the carry kept in a one-bit register between beats. The architectural result in the integer register is identical to a single-cycle RV32I implementation. Only the latency changes.

On BX-16 the beat is a halfword. A load word is two beats. The vector element width of SVM-16 matches the bus: eight or sixteen bits. A unit-stride vector load moves VLEN/16 halfwords when SEW is 16, and the bus can supply one halfword per beat.

BX-32 is the first member whose datapath, bus, and XLEN are all thirty-two bits. BX-64 widens XLEN itself to 64 and implements RV64I, together with the floating-point and vector extensions its profile names.

2.2 One decode for scalar and vector

Vector instructions are not a coprocessor opcode map glued beside the integer decode. They are OP-V in the standard encoding. The decoder classifies a parcel as integer, load, store, configuration, or vector arithmetic, and steers it. On the narrow machines the vector lanes are as narrow as the bus. A VLEN of 32 bits with eight-bit elements is four lanes, and four lanes of eight bits fit in a small FPGA without a second clock domain.

This is why a kernel written for SVM-8 is still RVV. The assembler emits `vsetvli` and `vadd.vv`. BX-32 will execute that parcel if the requested SEW is legal there. The ladder is a sequence of legal SEWs, not a sequence of source translators.

2.3 Latency you should publish

Publish, next to the profile card, the cycle count of the operations a programmer will feel.

Operation	BX-8	BX-16	BX-32
ALU on XLEN bits	4 beats	2 beats	1
Unit-stride beat	1 byte	1 halfword	1 word
`vsetvl`	1	1	1

Vector arithmetic latency is a parameter of the lane pipeline, not of XLEN. Keep `vsetvl` single-cycle. Configuration is how software discovers VLMAX, and hiding it in a long pipeline teaches the wrong lesson.

2.4 What moves upward unchanged

A strip-mined loop that only uses EEW=8 is legal on every machine in the book. A loop that uses EEW=16 is legal from BX-16 upward. A loop that uses EEW=32 or FP32 is legal from BX-32 upward. A loop that uses EEW=64 or FP64 is legal on BX-64.

The source does not change. The elaborated hardware refuses, at `vsetvl`, any SEW above its ELEN. That refusal is the entire compatibility story between profiles. There is no mode bit that widens a machine beyond the profile on its card.

Chapter 3

SVM Profiles and the Ratified Vector Extensions

3.1 The names

SVM stands for scalar, vector, and matrix. In this book the scalar part is the RISC-V integer hart, the vector part is a profile of RVV, and the matrix part is an optional custom extension called Xsvm. Xsvm is not implemented unless the elaboration parameter asks for it. The profiles themselves do not include matrix instructions.

Table 3.1: The ladder as a contract.

	BX-8	BX-16	BX-32	BX-64
Name	SVM-8	SVM-16	Zve32f	V or Zve64d
XLEN	32	32	32	64
Datapath	8	16	32	64
ELEN	8	16	32	64
VLEN	32	128	≥ 128	≥ 128
Legal EEW	8	8, 16	8, 16, 32	8, 16, 32, 64
FP	no	no	FP32	FP32 and FP64
Ratified claim	no	no	yes	yes

VLEN on BX-32 and BX-64 may be 256 or 512. The minimum is 128 because that is what the application extension requires, and because Zve32f implementations in this book are built to the same floor so that a BX-32 image can later be compared with a BX-64 image on the same vector length.

3.2 Why SVM-8 is not Zve32x

Zve32x requires element widths 8, 16, and 32, a minimum VLEN of at least ELEN, and the integer vector instruction set that goes with those widths. SVM-8 implements one of those widths. Reporting Zve32x from BX-8 would tell a compiler that `vsetvl` with SEW=32 is legal. It is not. The machine raises an illegal-instruction exception.

SVM-16 fails the same test one step higher. SEW=32 is illegal. SEW=8 and SEW=16 are legal. The machine also implements the saturating and widening integer operations used by sixteen-bit kernels, still under the OP-V encoding.

From BX-32 the claim is allowed. Zve32f adds single-precision floating point on top of the integer widths through 32. BX-64 claims V when VLEN is at least 128 and the double-precision vector operations are present; otherwise it claims Zve64d and says so.

3.3 The `misa` rule

On BX-8 and BX-16, `misa` reports RV32I and the extensions you have actually implemented among M, A, and the CSR set. It does not set the V bit. Software discovers SVM-8 or SVM-16 through a documented custom CSR, or through an illegal-instruction probe of SEW, whichever you publish in the profile card. Pick one and test it.

On BX-32, `misa` may set the bits for the extensions you implement, and the vector discovery path is the standard one for Zve32f. On BX-64 the V bit is set only if you implement V, not merely a subset you hope to finish later.

3.4 Xsvm

Two operations recur in the geometry the sixteen-bit machine is asked to run: a four-by-four matrix times a vector, and a three-element cross product. RVV expresses both as short sequences. BX-16 may also offer them as custom opcodes in the custom-0 or custom-1 space, under the name Xsvm.

If Xsvm is present it is an addition, not a substitute. A kernel that uses only standard OP-V must run on a BX-16 built without Xsvm. Document the opcodes in the same header that documents the discovery CSR. Do not alias them onto standard vector opcodes.

3.5 Fixed point without a private lane type

Sixteen-bit pixels and samples are ordinary SEW=16 integers. A product that needs more than sixteen bits is a widening multiply into a thirty-two-bit element group. A Q16.16 value, if software wants one, is that product followed by a shift. The hardware does not have a third element type for fixed point. Rounding and saturation use `vxrm` and `vxsat` where the specification already defines them, and only on the machines that implement those instructions.

Chapter 4

Vector State

4.1 The register file

Every machine in this book has thirty-two vector registers, numbered 0 through 31. Register 0 is also the mask. When an instruction is masked, bit i of `v0` says whether element i is active. Implementing a subset of the registers would change the encoding, so the file is always thirty-two deep even when each register is only thirty-two bits wide, as on BX-8. That file is 128 bytes. On BX-16, VLEN is 128 and the file is 512 bytes. Both sizes are small next to a video memory.

Registers are not grouped in the SVM-8 profile. LMUL stays 1. SVM-16 allows LMUL in $\{1, 2, 4, 8\}$ for widening destinations, because a sixteen-bit multiply that widens needs a thirty-two-bit place to land, and that place is a register group. If you would rather forbid LMUL greater than 1 on SVM-16, widening stores its high half in an explicit second destination named by the instruction you implement. Pick one, write it on the profile card, and do not mix them.

4.2 Configuration registers

The programmer-visible configuration is the standard one.

- `vtype` holds the selected element width, LMUL, and the tail and mask policy bits.
- `vl` holds the number of elements to process.
- `vstart` holds the element at which a partially completed instruction would resume. On these machines instructions either complete or trap before they write, and `vstart` is zero whenever software is allowed to read it. State that in the profile card. It is a legal simplification if you never resume a trapped vector instruction in the middle.
- `vxrm` and `vxsat` exist on any machine that implements fixed-point rounding or saturation. SVM-8 may omit them if it omits those instructions.

`vsetvl` and `vsetvli` are the only way to change `vtype` and `vl`. The instruction writes the chosen `vl` to the integer destination register as well as to the CSR. That write is how a strip-mined loop learns how many elements it was granted.

4.3 Legality at configuration time

The decoder of `vsetvl` checks SEW against ELEN and LMUL against the profile. An illegal request sets the `vill` bit of `vtype` and does not change `vl`, following the specification's rule for an invalid configuration. Subsequent vector instructions then trap as illegal. Do not leave `vill` clear and fail later inside a lane. Software cannot recover a clean meaning from a lane that has already consumed a beat of the bus.

On SVM-8, SEW must be 8 and LMUL must be 1.
On SVM-16, SEW must be 8 or 16.
On Zve32f, SEW must be 8, 16, or 32.
On V or Zve64d, SEW must be 8, 16, 32, or 64.

4.4 Tail and mask policy

The specification allows undisturbed and agnostic policies for inactive elements and for the tail past `vl`. Agnostic is easier in hardware: inactive lanes may write anything. Undisturbed is what a careful blit wants, because the tail of a scanline must stay what it was.

Implement undisturbed for the body of this book. It costs a read of the old destination element and a select. On an eight-bit lane that read is cheap. Publishing agnostic as well is optional and must be tested; a single policy, undisturbed, is the one the exercises assume.

4.5 Precise traps

A vector load that faults does so with `vstart` naming the first element that did not complete, or the instruction does not start the bus transaction until the whole access is known to be mapped. The second option is the one these machines take on BX-8 and BX-16, where the address map is a flat peripheral space without demand paging. BX-64, if it runs with a page table, uses the first option and a documented `vstart`. The profile card says which.

Chapter 5

The Instruction Classes

5.1 A minimum that is still RVV

Each profile implements a rectangle of the vector specification, not a sketch of it. Inside that rectangle every instruction you claim is the standard one: same encoding, same register numbers, same exception conditions. Outside the rectangle the instruction is illegal.

The rectangle grows as follows.

5.2 Configuration

Both `vsetvl` and `vsetvli` are required on every machine. `vsetivli` is required from SVM-16 upward. Immediate configuration is how small kernels avoid a spare integer register, and BX-8 may include it too if the decoder is already wide enough. If BX-8 omits it, the profile card says so.

5.3 Loads and stores

Unit-stride loads and stores of the legal element widths are required everywhere. The address is a byte address in the integer register, as in the specification. On BX-8 the bus moves one byte per beat, so a unit-stride load of `vl` elements occupies `vl` beats. On BX-16 a SEW=16 unit-stride load occupies `vl` halfword beats when the address is aligned, and two byte beats per element when it is not. Alignment faults follow the scalar policy of the hart: if scalar misaligned accesses are allowed, vector ones are allowed, and they take the extra beats.

Constant-stride loads are required from SVM-16 upward. Indexed loads are required from BX-32 upward. They are the gathers a texture or a sparse blit actually uses. Leaving them out of SVM-8 keeps the first load unit a counter and an address adder.

Segment loads are not required in any profile in this book. A compiler that emits them will trap. Say so next to the profile card.

5.4 Integer arithmetic

For every legal EEW the machine implements vector add, subtract, and, or, xor, and the shifts, in both vector-vector and vector-scalar forms. Compare and min/max are required from

SVM-16 upward. SVM-8 includes compare, because a mask is the natural way to clip a byte.

Widening multiply is required on SVM-16 for SEW=8 producing EEW=16, and for SEW=16 producing EEW=32. The thirty-two-bit destination is the reason SVM-16 allows a grouped destination even though ELEN is 16. Narrowing shifts are required on the same machine so that a widened product can return to a sixteen-bit pixel.

Saturating add and subtract are required on SVM-16 and on BX-32. SVM-8 includes saturating add only. Saturation sets `vxsat` when that CSR is implemented.

5.5 Reductions, masks, and permutes

An integer add reduction is required from SVM-16 upward. It is how a dot product of sixteen-bit samples finishes without a scalar loop over lanes. Slides by one element are required from SVM-16 upward. Gather-permute within a register is required from BX-32 upward.

Mask logical operations and mask population count are required wherever compare is required. The mask is `v0`. There is no second mask file.

5.6 Floating point

No floating-point vector instruction is legal on BX-8 or BX-16. The scalar hart on those machines also omits the F and D extensions, so a compiler for the replacement MSX or Amiga is an integer compiler plus the vector profile.

BX-32 implements the single-precision vector operations of Zve32f: add, subtract, multiply, fused multiply-add, min, max, and compare, plus the conversions between SEW=32 integers and FP32. BX-64 adds the double-precision set required by Zve64d, and the rest of V if that is the claim.

5.7 Custom geometry

Xsvm, when elaborated, adds two instructions in custom-0:

- a four-by-four matrix held in `v0`–`v3` times a vector in `vs2`, result in `vd`;
- a three-element cross product of `vs2` and `vs1` into `vd`.

Both operate at the current SEW and honour `vl`. They are illegal on a build that did not ask for Xsvm. They are illegal on BX-8 even if someone sets the parameter, because a four-by-four of bytes is not the operation the extension is for. The parameter is honoured from BX-16 upward only.

5.8 Instructions you will be tempted to skip

Division, square root, and the transcendental estimates are not in SVM-8 or SVM-16. They are optional on BX-32 and required on BX-64 only to the extent the claimed extension requires them. A reciprocal estimate with a Newton step in software is an acceptable published gap on BX-32 if the profile card lists it. It is not an acceptable gap if you have set the V bit.

Chapter 6

Lanes, Register Files, and Memory on an FPGA

6.1 How many lanes

A lane is a slice of the vector register file plus the functional units that operate on one element per cycle. The number of lanes does not change the architecture. It changes how many elements retire per cycle.

Machine	VLEN	Suggested lanes	Reason
BX-8	32	1 or 4	Four eight-bit lanes finish a register in one cycle.
BX-16	128	4	Four sixteen-bit lanes match a modest DSP column.
BX-32	128	4	One FP32 unit per lane is already a large board image.
BX-64	128 or 256	2 or 4	FP64 is the expensive unit; fewer lanes first.

Start with one lane on the first elaboration of each profile. Widen after the tests in Chapter 13 pass. A parameter `NLANES` that must divide VLEN/SEW for every legal SEW is the usual constraint. On SVM-16, VLEN is 128 and SEW may be 8, so eight elements of eight bits fill a register: `NLANES` should divide 8. Four is the comfortable choice.

6.2 The register file

Each lane owns a slice of every vector register. With four lanes and VLEN 128, a lane holds 32 bits of each register. Reads for a vector-vector instruction are two ports; the write back is a third. A mask read is a fourth, narrower port, or a bit collected from `v0`'s slice.

On an FPGA, implement the file as distributed memory for SVM-8 and as block RAM for SVM-16 and above if the port requirement fits the RAM primitive. If the primitive is dual-port and you need three accesses, bank the file by element and accept a one-cycle bubble when two reads and a write collide, or replicate the file. Replication is the smaller surprise on a teaching

design. State the choice in the profile card under “microarchitecture”, not under “architecture”. Software must not be able to tell, except by cycle count.

6.3 Chaining

Chaining means a consumer starts when the first element of its producer is ready, rather than when the whole vector has retired. It is worth building on BX-32 and BX-64, where floating-point latency is several cycles and kernels are long. It is not worth building on BX-8. A four-element register finishes before a chain controller pays for itself.

The in-order rule for BX-8 and BX-16 is: one vector instruction in the arithmetic unit at a time, loads and arithmetic allowed to overlap only when the load target is not a source of the arithmetic instruction. Scoreboard those three register numbers. That is enough hazard logic for the first board.

6.4 Memory width

Match the vector memory port to the bus you already have for the peripherals, not to VLEN.

On the MSX-compatible machine the bus is eight bits. The vector load unit is a byte counter. On the Amiga-compatible machine the bus is sixteen bits and is shared with video and blitter masters. The vector load unit requests halfwords through the same arbiter as those masters. Do not add a private wide path from the vector file to memory on BX-16; it would no longer be the bus an Amiga-compatible board has.

On BX-32 and BX-64 a dedicated wider port is appropriate, because those machines are not standing in for an eight-bit or sixteen-bit chipset. Even there, size the port to a power of two beats of the element width you care about, and measure utilisation with the fill test before widening it again.

6.5 What to read in other people’s cores

Ara shows a lane-based file, a central sequencer, and a slide unit, on a full application vector extension, in a process where a lane’s area is a published number. Spatz shows that a small file and a shared scratchpad can be enough when the data are already local. Vicuna shows a coprocessor interface and a configurable set of pipelines aimed at Zve32x.

None of them is a Bluespec description of SVM-8, and none of them is an MSX or Amiga bus. Read them for the shape of a lane and for the list of hazards. Write your own rules against the profile card. Importing a large SystemVerilog vector unit beside an eight-bit peripheral fabric will not produce a machine that can replace an MSX.

Chapter 7

BX-8: An MSX-Compatible Machine with SVM-8

7.1 The board-level picture

BX-8 is a single FPGA image that can replace an MSX in the narrow sense of Chapter 4. The parts a cartridge and a display see are the slot signals, the video display processor, the sound generator, the keyboard and joystick ports, and the memory map. The part that executes instructions is an RV32I hart. Its data path is eight bits. Its vector unit is SVM-8.

A word in an integer register is still thirty-two bits. The program counter advances by four when the parcel is an ordinary instruction, because the encoding is thirty-two bits even though the data bus is not. Instruction memory may be a separate thirty-two-bit port. Data memory and I/O share the eight-bit bus with the video processor. That split is the usual way to keep fetch off the peripheral bus, and it does not change the architectural width of a load.

7.2 The hart

Fetch reads a thirty-two-bit parcel. Decode recognises RV32I and the OP-V opcodes of SVM-8. Execute for an integer instruction runs four byte beats through an eight-bit ALU, with a carry flop between beats. Writeback commits the whole register or, on a fault, commits nothing.

There is no separate Z80 or R800 pipeline. Software written for those processors runs only if you have written an interpreter or a translator in RV32I. The peripherals do not know which interpreter is running. They answer to port writes in the MSX map.

The multiplier, if you include the M extension, is a sequenced eight-bit multiplier, not a DSP that produces a thirty-two-bit product in one cycle. Leaving M out of the first image is acceptable. Vector byte adds cover the inner loops that would otherwise have wanted a fast scalar multiply.

7.3 SVM-8

ELEN = 8, VLEN = 32, thirty-two registers, LMUL = 1.
Legal SEW = 8 only.
Required: `vsetvl`, `vsetvli`, unit-stride load and store,
add, subtract, logic, shifts, compare, saturating add, splat (`vmv.v.x`),

vector-vector and vector-scalar forms.
Undisturbed tail and mask.
No floating point, no indexed load, no Xsvm.

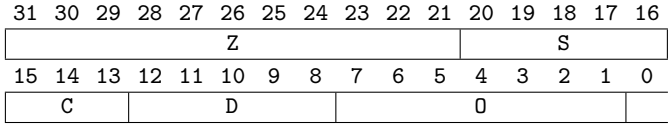
Four elements fill a register. With four lanes, a vector add retires in one cycle after the sources are read. With one lane, it retires in four. Either organisation passes the same architectural tests.

The load unit takes the base address from an integer register, checks that `vl` bytes lie in a mapped region, and then consumes `vl` bus beats when the arbiter grants them. Video refresh has priority on that arbiter. A vector load that loses the bus for a scanline simply waits. That wait is the MSX-compatible behaviour worth preserving: the picture does not tear because a copy loop held the bus.

7.4 Opcode map for SVM-8

All encodings are 32-bit. The diagrams use the approved two-line rigid matrix with single-letter field codes. Only the forms legal under ELEN = 8 and LMUL = 1 are shown. Every other OP-V encoding traps.

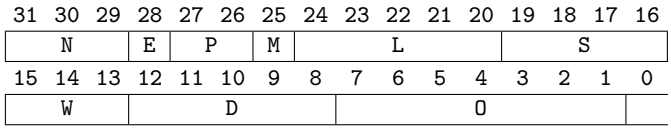
7.4.1 Configuration — `vsetvli`



Z = zimm[10:0] (vtype immediate)
S = rs1
C = 111 (OPCFG)
D = rd
O = 1010111

`vsetvl` replaces the zimm field with rs2. `vsetivli` (if implemented) uses a 5-bit immediate in place of rs1.

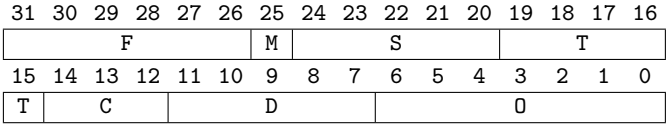
7.4.2 Unit-stride load — `vle8.v`



N = nf (normally 000)
E = mew
P = mop (00 = unit-stride)
M = vm
L = lumop (00000 = unit-stride)
S = rs1 (base)
W = width (000 = 8-bit)
D = vd
O = 0000111

Store form `vse8.v` uses major opcode 0100111 and vs3 in the D field.

7.4.3 Integer arithmetic (OPIVV / OPIVX)



F = funct6
M = vm
S = vs2
T = vs1 (vv) or rs1 (vx)
C = funct3 (000 = OPIVV, 100 = OPIVX)
D = vd
O = 1010111

Instruction	funct6	form
`vadd`	000000	vv / vx
`vsub`	000010	vv / vx
`vrsub`	000011	vx only
`vand`	001001	vv / vx
`vor`	001010	vv / vx
`vxor`	001011	vv / vx
`vsll`	100101	vv / vx
`vsrl`	101000	vv / vx
`vsra`	101001	vv / vx
`vsadd`	100001	vv / vx
`vssub`	100011	vv / vx (optional)

7.4.4 Compare (mask result)

Same layout as integer arithmetic. funct3 = 000 (OPIVV) or 100 (OPIVX). Result is written to the mask register `v0` when vd = 0, or to the named mask destination.

Instruction	funct6	form
`vmseq`	011000	vv / vx
`vmsne`	011001	vv / vx
`vmsltu`	011010	vv / vx
`vmslt`	011011	vv / vx
`vmsleu`	011100	vv / vx
`vmsle`	011101	vv / vx
`vmsgtu`	011110	vx only
`vmsgt`	011111	vx only

7.4.5 Move and splat

Instruction	encoding notes
`vmv.v.v`	funct6 = 010111, vs1 = vs2, OPIVV
`vmv.v.x`	funct6 = 010111, OPIVX
`vmv.v.i`	funct6 = 010111, OPIVI
`vmv.x.s`	whole-register scalar extract
`vmv.s.x`	whole-register scalar insert

7.4.6 Mask logicals

Instruction	funct6
`vmand`	011001
`vmnand`	011101
`vmandn`	011000
`vmor`	011010
`vmnor`	011110
`vmorn`	011100
`vmxor`	011011
`vmxnor`	011111
`vcpop`	010000 (OPMVV)
`vfirst`	010001 (OPMVV)

No widening, no narrowing, no stride, no indexed, no reduction, no slide, no floating-point, and no custom geometry opcodes are legal on SVM-8.

7.5 Assembler kernels that must run

These routines are the acceptance set for every BX-8 image. They use only the opcodes listed above and must produce bit-identical results to a scalar reference model.

7.5.1 fill8 — fill a region with a byte

```
# a0 = n, a1 = fill byte, a2 = dst
fill8:
    beqz a0, 2f
1: vsetvli t0, a0, e8, m1, tu, mu
    vmv.v.x v1, a1
    vse8.v v1, (a2)
    add a2, a2, t0
    sub a0, a0, t0
    bnez a0, 1b
2: ret
```

7.5.2 copy8 — unit-stride byte copy

```
# a0 = n, a1 = src, a2 = dst
copy8:
    beqz a0, 2f
1: vsetvli t0, a0, e8, m1, tu, mu
```

```
    vle8.v v1, (a1)
    vse8.v v1, (a2)
    add a1, a1, t0
    add a2, a2, t0
    sub a0, a0, t0
    bnez a0, 1b
2: ret
```

7.5.3 mask8 — selective fill under a generated mask

```
# a0 = n, a1 = fill, a2 = dst, a3 = threshold
mask8:
    beqz a0, 2f
1: vsetvli t0, a0, e8, m1, tu, mu
    vle8.v v1, (a2)
    vmsltu.vx v0, v1, a3
    vmv.v.x v2, a1
    vse8.v v2, (a2), v0.t
    add a2, a2, t0
    sub a0, a0, t0
    bnez a0, 1b
2: ret
```

7.5.4 add8 — element-wise byte add with saturation

```
# a0 = n, a1 = src0, a2 = src1, a3 = dst
add8:
    beqz a0, 2f
1: vsetvli t0, a0, e8, m1, tu, mu
    vle8.v v1, (a1)
    vle8.v v2, (a2)
    vsadd.vv v3, v1, v2
    vse8.v v3, (a3)
    add a1, a1, t0
    add a2, a2, t0
    add a3, a3, t0
    sub a0, a0, t0
    bnez a0, 1b
2: ret
```

7.6 What you elaborate

```
typedef enum { SVM8, SVM16, ZVE32F, V64 } Profile;
Profile profile = SVM8;
Integer elen = 8;
Integer vlen = 32;
Integer lanes = 1; // or 4
Bool xsvm = False;
Bool vsetivli = False; // True only if decoder already has it
```

The same package, with different parameters, is BX-16 and the later machines. BX-8 is the parameter set in which most of the package is illegal-instruction logic rather than arithmetic.

Chapter 8

BX-16: An Amiga-Compatible Machine with SVM-16

8.1 The board-level picture

BX-16 can replace an Amiga in the same practical sense that BX-8 can replace an MSX. The bus is sixteen bits wide. Several masters share it: the hart, a blitter-class block mover, video fetch, and audio DMA. Chip memory and a larger local memory sit on that bus. The display timing can follow a programmed list of fetches, in the role a copper list fills on the original machine. The hart that runs the list is RV32I with a sixteen-bit data path.

Integer registers remain thirty-two bits. A load word takes two halfword beats. Instruction fetch is again allowed its own port so that video DMA does not stall the decode of every parcel.

8.2 SVM-16

ELEN = 16, VLEN = 128, thirty-two registers.
Legal SEW = 8 or 16.
LMUL may be greater than 1 only for a widening destination.
Required, beyond SVM-8: strides, widening multiply,
narrowing shift, saturating add and subtract,
integer add reduction, slide by one, `vsetivli`.
No floating point. Xsvm optional.

A register holds sixteen elements of eight bits or eight elements of sixteen bits. That is enough for a short span of a scanline or a frame of samples. Longer spans strip-mine, and the bus arbiter inserts the video and audio masters between strips. Write the arbiter so that a maximum latency for the audio master is enforced. A vector copy that can lock out audio is not an Amiga-compatible bus, however correct its RVV tests are.

8.3 Widening, not a private fixed-point format

A sixteen-bit multiply that keeps the full product uses `vwmul.vv`. The destination elements are thirty-two bits wide, in a register group. A following narrowing shift brings a saturated sixteen-bit pixel back to SEW=16. Software that wants a Q16.16 convention shifts by sixteen. The lane does not test a mode bit named Q16.16.

This is the operation a blitter-class scaler and a small transform both need. Implementing it as a special coprocessor instruction would make the kernel unreadable on BX-32. Implementing it as `vwmul` means the same source runs on Zve32f.

8.4 Xsvm

When the elaboration parameter `xsvm` is true, two custom-0 instructions are decoded.

The matrix instruction reads four vector registers starting at a fixed base, `v8` through `v11`, as the rows of a four-by-four matrix, multiplies them by `vs2`, and writes `vd`. Using `v8` rather than `v0` keeps the mask register free. The cross-product instruction reads two registers and writes one, three elements, at the current SEW, with `vl` forced to a minimum of three or the instruction is illegal.

Both instructions are undisturbed outside `vl`. Both are absent from the decoder when `xsvm` is false, and the parcels trap. The exercises that must pass on every BX-16 image do not use them.

8.5 Opcode map for SVM-16

Everything legal on SVM-8 remains legal. The additional classes required by SVM-16 are shown with the same rigid two-line matrix. SEW may be 8 or 16. The complete cross-profile contract and full opcode summary appear after the four machines, in Chapter 11.

8.5.1 Configuration

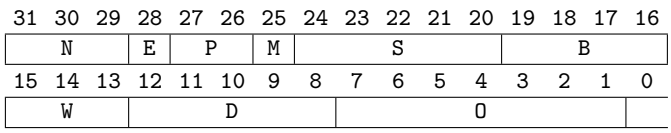
Identical to SVM-8. `vsetivli` is now required.

8.5.2 Unit-stride load/store (EEW = 8 and 16)

Same layout as SVM-8. Width field:

- 000 = 8-bit (`vle8.v`/`vse8.v`)
- 101 = 16-bit (`vle16.v`/`vse16.v`)

8.5.3 Constant-stride load — `vlse16.v`



N = nf
E = mew
P = mop (10 = strided)
M = vm
S = rs2 (stride)
B = rs1 (base)
W = width (101 = 16-bit)
D = vd
O = 0000111

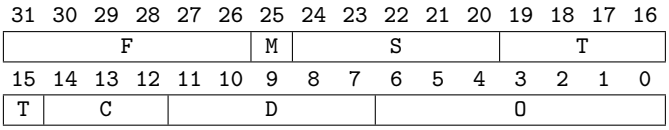
Store form uses major opcode 0100111.

8.5.4 Integer arithmetic (SEW = 8, 16)

Same OP-V layout as SVM-8. Additional instructions:

Instruction	funct6	notes
`vminu/vmin`	000100 / 000101	vv / vx
`vmaxu/vmax`	000110 / 000111	vv / vx
`vsaddu/vsadd`	100000 / 100001	saturating
`vssubu/vssub`	100010 / 100011	saturating

8.5.5 Widening multiply and narrowing



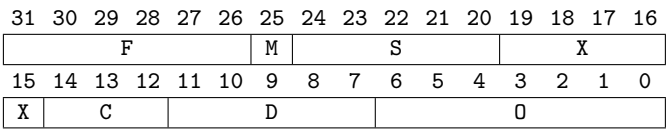
F = funct6
M = vm
S = vs2
T = vs1 / rs1
C = funct3 (OPIVV / OPIVX)
D = vd (group for widening)
O = 1010111

Instruction	funct6	width rule
`vwmul`	111011	SEW → 2×SEW
`vwmulu`	111000	unsigned
`vwmulsu`	111010	mixed
`vnsrl`	101100	narrowing shift
`vnsra`	101101	narrowing arithmetic
`vnclipu/vnclip`	101110 / 101111	narrowing clip

8.5.6 Reduction and slide

Instruction	funct6 / notes
`vredsum.vs`	000000 (OPMVV)
`vslide1up.vx`	001110 (OPIVX)
`vslide1down.vx`	001111 (OPIVX)

8.5.7 Optional Xsvm (custom-0)



F = custom funct6
M = vm
S = vs2
X = unused / fixed
C = custom funct3
D = vd
O = 0001011 (custom-0)

Matrix and cross-product are distinguished by the custom funct6 / funct3 values documented in the profile card. They are illegal when `xsvm = False`.

8.6 Assembler kernels that must run

In addition to every kernel of Chapter 7, the following must pass on every BX-16 image (Xsvm kernels are excluded from the travelling suite).

8.6.1 sew16 — halfword fill

```
# a0 = n (elements), a1 = fill halfword, a2 = dst
sew16:
    beqz a0, 2f
1: vsetvli t0, a0, e16, m1, tu, mu
    vmv.v.x v1, a1
    vse16.v v1, (a2)
    slli t1, t0, 1
    add a2, a2, t1
    sub a0, a0, t0
    bnez a0, 1b
2: ret
```

8.6.2 vwmul16 — widening multiply then narrow with saturation

```
# a0 = n, a1 = src0 (SEW16), a2 = src1, a3 = dst (SEW16)
vwmul16:
    beqz a0, 2f
1: vsetvli t0, a0, e16, m1, tu, mu
    vle16.v v2, (a1)
    vle16.v v4, (a2)
    vwmul.vv v8, v2, v4
    vsetvli t0, a0, e16, m1, tu, mu
    vnclip.wi v1, v8, 0
    vse16.v v1, (a3)
    slli t1, t0, 1
    add a1, a1, t1
    add a2, a2, t1
    add a3, a3, t1
    sub a0, a0, t0
    bnez a0, 1b
2: ret
```

8.6.3 stride16 — constant-stride gather of halfwords

```
# a0 = n, a1 = src base, a2 = stride (bytes), a3 = dst
stride16:
    beqz a0, 2f
1: vsetvli t0, a0, e16, m1, tu, mu
    vlse16.v v1, (a1), a2
    vse16.v v1, (a3)
    slli t1, t0, 1
    add a3, a3, t1
    sub a0, a0, t0
    bnez a0, 1b
2: ret
```

8.6.4 redsum16 — integer reduction

```
# a0 = n, a1 = src (SEW16), a2 = scalar accumulator out
redsum16:
    vsetvli t0, a0, e16, m1, tu, mu
    vle16.v v1, (a1)
    vmv.v.i v2, 0
    vredsum.vs v2, v1, v2
    vmv.x.s a2, v2
    ret
```

8.7 One hart

Scalar parcels and vector parcels share decode, the scoreboard, and writeback. The blitter, if it is a bus master, is not a second hart and does not fetch OP-V. It is a peripheral with a pointer, a count, and a small arithmetic step, programmed by stores to its registers. New software may ignore it and use SVM-16 instead. Old software that you have translated may still program it. Keeping both is how the machine remains Amiga-compatible while the new kernels are ordinary RVV.

8.8 Elaboration

```
Profile profile = SVM16;
Integer elen = 16;
Integer vlen = 128;
Integer lanes = 4;
Bool xsvm = False; // True only on a geometry image
```

Four lanes of sixteen bits retire four elements per cycle. A full SEW=16 register then takes two arithmetic cycles, plus the read and write of the file. Publish that number. Programmers of audio mixing will plan their strip length around it.

Chapter 9

BX-32: Embedded Floating Point with Zve32f

9.1 The first ratified claim

BX-32 is an RV32 hart whose data path is thirty-two bits, with the scalar extensions you need for a hosted compiler, and with Zve32f. This is the first machine in the book that is allowed to tell a standard toolchain “vector, with single-precision float”. The discovery path is the standard one. The custom CSR used by SVM-8 and SVM-16 may remain, returning a code that means “superset of SVM-16”, but `misa` and the vector CSRs are authoritative.

ELEN = 32. VLEN is at least 128.
Legal SEW = 8, 16, and 32.
FP32 vector arithmetic, conversions, and fused multiply-add.
Indexed loads and stores.
Integer behaviour of SVM-16 remains legal.

A loop assembled for SVM-8, using only EEW=8, runs. A loop assembled for SVM-16, using EEW=16 and `vwmul`, runs. A loop that sets SEW=32 runs for the first time.

9.2 Floating point in the lane

Each lane that supports FP32 contains a single-precision adder and multiplier, or a fused unit if the FPGA’s DSP blocks map cleanly onto one. Fused multiply-add is required by the extension. Rounding follows the dynamic rounding mode of the hart.

Do not share one FP unit across all lanes by silent time-multiplexing unless the cycle count is in the profile card. A programmer who reads Zve32f will expect four lanes to retire four FP32 elements per cycle once the pipeline is full. If your first image has one FP unit and four integer lanes, say “integer lanes: 4, FP lanes: 1” on the card, and do not claim that the peak is four times the FP unit.

9.3 Where SVM-16 software feels the change

The change a programmer notices is not the wider bus. It is that SEW=32 no longer traps, and that a matrix kernel can stay in floating point instead of in widened integers. Xsvm, if it

is still elaborated, must use the same SEW as the surrounding code. A four-by-four of FP32 values in `v8`–`v11` is the useful form. The integer form remains for SVM-16 compatibility.

Prefer, in new BX-32 software, the standard sequence of fused multiply-adds over Xsvm. Keep Xsvm for kernels you have measured, and for comparison with the BX-16 geometry image.

9.4 Opcode map for Zve32f (BX-32)

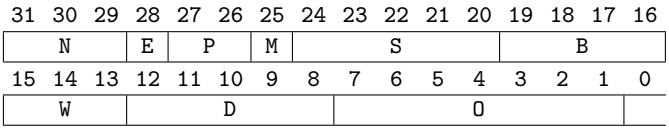
All opcodes of SVM-16 remain legal. The additional classes required by Zve32f use the same rigid two-line matrix. SEW may be 8, 16 or 32.

9.4.1 Configuration

Identical to SVM-16.

9.4.2 Unit-stride, stride and indexed load/store

Unit-stride and stride use the layouts already shown. Indexed load (`vluxei32.v` / `vloxei32.v`):



N = nf
E = mew
P = mop (01 = indexed unordered, 11 = ordered)
M = vm
S = vs2 (index vector)
B = rs1 (base)
W = width (110 = 32-bit)
D = vd
O = 0000111

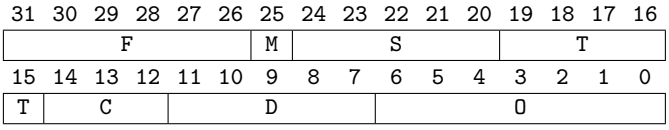
Store forms use major opcode 0100111.

9.4.3 Integer arithmetic (SEW = 8, 16, 32)

Same OP-V layout. All integer instructions of SVM-16 plus the SEW=32 variants and gather:

Instruction	notes
`vrgather.vv/vx`	register gather
`vcompress.vm`	compress under mask

9.4.4 Floating-point arithmetic (SEW = 32)



F = funct6
M = vm
S = vs2
T = vs1 / rs1 / fs1
C = funct3 (001 = OPFVV, 101 = OPFVF)
D = vd
O = 1010111

Instruction	funct6	form
`vfadd`	000000	vv / vf
`vfsub`	000010	vv / vf
`vfmul`	100100	vv / vf
`vfmacc`	101100	vv / vf
`vfnmacc`	101101	vv / vf
`vfmsac`	101110	vv / vf
`vfnmsac`	101111	vv / vf
`vfmadd`	101000	vv / vf
`vfnmadd`	101001	vv / vf
`vfmsub`	101010	vv / vf
`vfnmsub`	101011	vv / vf
`vfmin`	000100	vv / vf
`vfmax`	000110	vv / vf
`vfsqrt`	010011	v
`vfsgnj*`	001000..001010	vv / vf
`vfcvt.*`	01000x	conversions
`vmfeq..`	011000..	FP compare

9.4.5 Mask and move

Full mask logical set of SVM-16 plus `vfmv.v.f`, `vfmv.f.s`, `vfmv.s.f`.

Xsvm remains optional and is not part of the Zve32f claim.

9.5 Assembler kernels that must run

In addition to every kernel of Chapters 7 and 8, the following must pass.

9.5.1 sew32 — word fill

```
# a0 = n, a1 = fill word, a2 = dst
sew32:
    beqz a0, 2f
1: vsetvli t0, a0, e32, m1, tu, mu
    vmv.v.x v1, a1
    vse32.v v1, (a2)
    slli t1, t0, 2
    add a2, a2, t1
    sub a0, a0, t0
    bnez a0, 1b
2: ret
```

9.5.2 fma32 — fused multiply-add strip

```
# a0 = n, a1 = a, a2 = b, a3 = c (dst = a*b + c)
fma32:
    beqz a0, 2f
1: vsetvli t0, a0, e32, m1, tu, mu
    vle32.v v1, (a1)
    vle32.v v2, (a2)
    vle32.v v3, (a3)
    vfmacc.vv v3, v1, v2
    vse32.v v3, (a3)
    slli t1, t0, 2
    add a1, a1, t1
    add a2, a2, t1
    add a3, a3, t1
    sub a0, a0, t0
    bnez a0, 1b
2: ret
```

9.5.3 index32 — indexed load (gather)

```
# a0 = n, a1 = base, a2 = index vector (bytes), a3 = dst
index32:
    beqz a0, 2f
1: vsetvli t0, a0, e32, m1, tu, mu
    vle32.v v2, (a2)
    vluxei32.v v1, (a1), v2
    vse32.v v1, (a3)
    slli t1, t0, 2
    add a2, a2, t1
    add a3, a3, t1
    sub a0, a0, t0
    bnez a0, 1b
2: ret
```

9.6 Two harts

A second RV32 hart is optional. It shares memory through an arbiter you already know how to write, because BX-16 has several masters. The second hart is for a concurrent system, not a different instruction set. Both harts, if both are present, implement Zve32f or the second is a scalar only and the profile card says so. Do not build one hart with FP32 vector and another with a private opcode map.

Local scratchpads are ordinary memories in the address map, marked cacheable or not according to your platform. They are not a third address space with a private load instruction. Vector loads to them use the same unit-stride encoding as vector loads to the rest of memory.

Chapter 10

BX-64: Application Vectors with V and Zve64d

10.1 The top of the ladder

BX-64 widens XLEN to 64. The base is RV64I. The vector unit is Zve64d or the full application extension V. ELEN is 64. VLEN is at least 128, and 256 is the recommended elaboration when the block RAM budget allows a file of thirty-two registers times 256 bits, which is one kilobyte.

> Legal SEW = 8, 16, 32, and 64.
> FP32 and FP64, including fused multiply-add.
> If the claim is V: every instruction V requires, VLEN at least 128,
> and the V bit set in `misa`.
> If the claim is Zve64d: do not set the V bit.

The second sentence is the one that keeps the ladder honest. A machine that implements double-precision vector arithmetic and not yet the whole of V is a useful machine. It is not a V machine.

10.2 What arrives with sixty-four-bit elements

SEW=64 is the element width a double-precision value needs, and the element width a widened thirty-two-bit product needs if you choose to widen that far. Both appear in geometry and in audio rate conversion. The lane's integer ALU grows a sixty-four-bit path, which on a sixteen-bit or thirty-two-bit DSP primitive is a short sequence of partial adds. That sequence is internal. The architectural latency is a published integer.

Masks, slides, and reductions at SEW=64 are required. Indexed accesses are required. `vstart` becomes architecturally visible if the machine has a page table and a vector load can fault on a later page than it started on. The simple machines avoided this by checking the whole access first. A sixty-four-bit hart with virtual memory cannot always do that cheaply. Implement `vstart`, test a fault on element 0 and a fault on a middle element, and document that restart is precise.

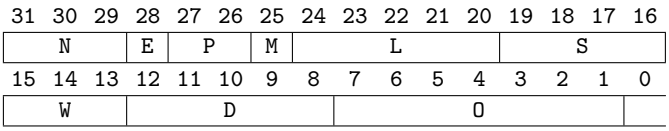
10.3 Opcode map for Zve64d / V (BX-64)

All opcodes of Zve32f remain legal. The additional element widths and FP64 operations use the same rigid two-line matrix. SEW may be 8, 16, 32 or 64. Every major class has an explicit entry diagram below or inherits the diagram from the lower profile.

10.3.1 Configuration

Identical to the lower profiles (see Chapter 7).

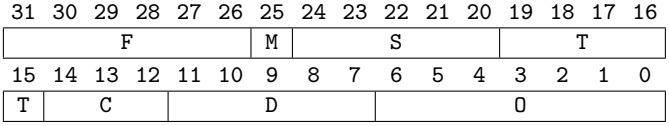
10.3.2 Unit-stride load at SEW=64 — `vle64.v`



N = nf
E = mew
P = mop (00 = unit-stride)
M = vm
L = lumop (00000)
S = rs1 (base)
W = width (111 = 64-bit)
D = vd
O = 0000111

Stride and indexed forms keep the same layout; only the mop and width fields change (width = 111 for EEW=64).

10.3.3 Integer / FP arithmetic (OP-V) at any SEW including 64



F = funct6
M = vm
S = vs2
T = vs1 / rs1 / fs1
C = funct3 (OPIVV / OPIVX / OPFVV / OPFVF ...)
D = vd
O = 1010111

All integer instructions of Zve32f are legal at SEW=64. All FP32 instructions remain legal; their FP64 counterparts use the same funct6 values with SEW=64.

Class	examples	notes
Integer	`vadd`, `vwmul`, `vredsum`, `vrgather`	full set
FP32	`vfadd`, `vfmacc`, `vfcvt`	from BX-32
FP64	`vfadd`, `vfmacc`, `vfcvt` at SEW=64	required for Zve64d/V

If the claim is the full application extension V, every additional instruction required by the V specification must be present and the V bit set in `misa`. A Zve64d implementation omits the V bit and may omit instructions that V adds beyond Zve64d.

10.3.4 Mask and move

Full mask set plus the FP move variants for both single and double.

Xsvm must be left off on any image that claims V.

10.4 Assembler kernels that must run

In addition to every kernel of the lower profiles, the following must pass on BX-64.

10.4.1 sew64 — double-word fill

```
# a0 = n, a1 = fill, a2 = dst
sew64:
    beqz a0, 2f
1: vsetvli t0, a0, e64, m1, tu, mu
    vmv.v.x v1, a1
    vse64.v v1, (a2)
    slli t1, t0, 3
    add a2, a2, t1
    sub a0, a0, t0
    bnez a0, 1b
2: ret
```

10.4.2 fma64 — double-precision fused multiply-add

```
# a0 = n, a1 = a, a2 = b, a3 = c (dst = a*b + c)
fma64:
    beqz a0, 2f
1: vsetvli t0, a0, e64, m1, tu, mu
    vle64.v v1, (a1)
    vle64.v v2, (a2)
    vle64.v v3, (a3)
    vfmacc.vv v3, v1, v2
    vse64.v v3, (a3)
    slli t1, t0, 3
    add a1, a1, t1
    add a2, a2, t1
    add a3, a3, t1
    sub a0, a0, t0
    bnez a0, 1b
2: ret
```

10.4.3 index64 — indexed load at SEW=64

```
# a0 = n, a1 = base, a2 = index vector, a3 = dst
index64:
    beqz a0, 2f
1: vsetvli t0, a0, e64, m1, tu, mu
```

```
    vle64.v v2, (a2)
    vluxei64.v v1, (a1), v2
    vse64.v v1, (a3)
    slli t1, t0, 3
    add a2, a2, t1
    add a3, a3, t1
    sub a0, a0, t0
    bnez a0, 1b
2: ret
```

10.5 Two harts, local memory, DMA

The organisation that justifies a second hart is the one the wide machines are for: one hart runs the operating system and short vector work, the other runs a longer kernel from a local memory. Both are RV64 with the same vector claim. A DMA engine moves blocks into that local memory. The DMA is a bus master programmed by stores, in the same style as the blitter on BX-16. It does not fetch vector instructions.

Local memory is mapped. Vector loads and stores name it with ordinary addresses. A kernel that ran on BX-32 in SEW=32 runs here in SEW=32 without change. A kernel that wants FP64 sets SEW=64 and uses the double-precision opcodes.

10.6 Xsvm on this machine

Leave Xsvm off on the image you use to claim V. A standard toolchain will not emit it, and a custom opcode in the middle of an otherwise clean V implementation confuses every test suite you did not write. A second elaboration, with Xsvm on, is how you compare a four-by-four custom instruction with the fused-multiply sequence. Publish both cycle counts. Keep the V-claiming image free of custom geometry.

Chapter 11

One Bluespec Description, Four Profiles

11.1 Complete opcode contract

After the four machines the architectural rectangle is fixed. The table is the single entry point for "which opcodes exist on which core".

Class / feature	SVM-8 (BX-8)	SVM-16 (BX-16)	Zve32f (BX-32)	Zve64d/V (BX-64)
`vsetvl/vsetvli`	yes	yes	yes	yes
`vsetivli`	opt.	yes	yes	yes
Unit-stride ld/st	EEW=8	8,16	8–32	8–64
Constant-stride	—	yes	yes	yes
Indexed ld/st	—	—	yes	yes
Segment ld/st	no	no	no	no
Integer arith (add/logic/shift)	yes	yes	yes	yes
min/max	—	yes	yes	yes
Compare + mask logicals	yes	yes	yes	yes
Saturating add/sub	vsadd only	yes	yes	yes
Widening multiply / narrow	—	yes	yes	yes
Reduction / slide1	—	yes	yes	yes
Gather / compress	—	—	yes	yes
Moves / splat (`vmv.*`)	yes	yes	yes	yes
FP32 vector + FMA	—	—	yes	yes
FP64 vector + FMA	—	—	—	yes
Division / sqrt / estimates	no	no	opt.	per claim
LMUL > 1	no	widen only	yes	yes
Xsvm (custom-0)	no	opt.	opt.	off on V
Tail policy	und.	und.	und.	und.

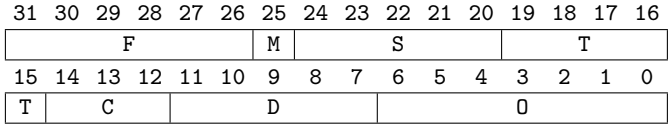
The bit-field diagrams that appear in Chapters 7–10 are the concrete encodings for each class. The tables below list every mnemonic the corresponding profile must implement. The Bluespec package that follows elaborates exactly the cells marked "yes" (or "opt." when the corresponding parameter is true) and turns every other cell into an illegal-instruction path.

11.2 Aggregated opcode tables (ISA)

Canonical bit layouts are shown once, width-aligned. Profile columns then list only the mnemonics (and the few constants that differ). Detailed per-machine diagrams remain in Chapters 7–10; this section is the compact academic summary.

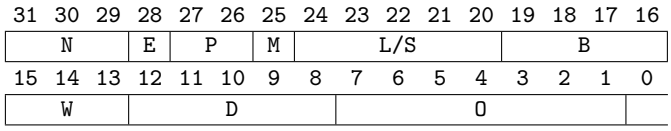
11.2.1 Canonical layouts (32-bit, two equal-width half-words)

OP-V arithmetic / compare / move / FP



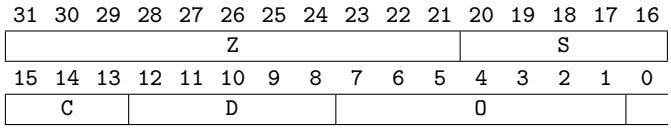
F = funct6 M = vm
S = vs2 T = vs1 / rs1 / fs1
C = funct3 D = vd
O = 1010111 (OP-V)

Unit-stride / stride / indexed load (store substitutes opcode 0100111)



N = nf E = mew
P = mop (00 unit, 10 stride, 01/11 indexed) M = vm
L/S = lumop or vs2 (index) B = rs1 (base)
W = width (000=8, 101=16, 110=32, 111=64) D = vd / vs3
O = 0000111 (load)

Configuration (`vsetvli`)



Z = zimm[10:0] S = rs1
C = 111 D = rd
O = 1010111

11.2.2 Mnemonic coverage by profile

Group	SVM-8	SVM-16	Zve32f	Zve64d/V
Configuration	`vsetvl/i`	+ `vsetivli`	same	same
Unit-stride	e8	e8,e16	e8–e32	e8–e64
Stride	—	e8,e16	e8–e32	e8–e64
Indexed	—	—	e8–e32	e8–e64
Integer arith	add/sub/logic/shift	+min/max	same	+SEW64
Saturating	vsadd	full set	full	full
Widening / narrow	—	vwmul, vnclip…	same	same
Compare + mask	full	full	full	full
Reduction / slide1	—	yes	yes	yes
Gather / compress	—	—	yes	yes
Moves / splat	`vmv.*`	same	+FP moves	same
FP32 + FMA	—	—	full	full
FP64 + FMA	—	—	—	full
Xsvm	no	opt.	opt.	off on V

11.2.3 Explicit summary — SVM-8 (BX-8)

Required	Notes
`vsetvl`, `vsetvli`	`vsetivli` optional
`vle8.v`, `vse8.v`	unit-stride only
`vadd/sub.vv/vx`, `vrsub.vx`	
`vand/vor/vxor.vv/vx`	
`vsll/vsrl/vsra.vv/vx`	
`vsadd.vv/vx`	`vssub` optional
`vmseq/ne/ltu/lt/leu/le.vv/vx`, `vmsgtu/gt.vx`	
mask logicals, `vcpop.m`, `vfirst.m`	
`vmv.v.v/x/i`, `vmv.x.s`, `vmv.s.x`	

11.2.4 Explicit summary — SVM-16 (BX-16)

Everything in SVM-8, plus:

Required	Notes
`vsetivli`	now required
`vle16.v`, `vse16.v`	
`vlse8/16.v`, `vsse8/16.v`	constant-stride
`vminu/min/maxu/max.vv/vx`	
`vsaddu/vsadd/vssubu/vssub`	full saturating set
`vwmul/u/su.vv/vx`	widening
`vnsrl/vnsra`, `vnclipu/vnclip`	narrowing
`vredsum.vs`	reduction
`vslide1up.vx`, `vslide1down.vx`	
Xsvm matrix / cross-product	optional (custom-0)

11.2.5 Explicit summary — Zve32f (BX-32)

Everything in SVM-16, plus:

Required	Notes
`vle32.v`, `vse32.v`	
`vluxei*`, `vloxei*`, indexed stores	
`vrgather.vv/vx`, `vcompress.vm`	
`vfadd/sub/mul.vv/vf`	FP32
`vfmacc/fnmacc/fmsac/fnmsac`	FMA family
`vfmadd/fnmadd/fmsub/fnmsub`	
`vfmin/max`, `vfsqrt`, `vfsgnj*`	
`vfcvt.*`, `vmfeq/ne/lt/le/gt/ge`	
`vfmv.v.f`, `vfmv.f.s`, `vfmv.s.f`	

11.2.6 Explicit summary — Zve64d / V (BX-64)

Everything in Zve32f, plus SEW = 64 and FP64. If the image claims full V, the entire V instruction set is required and the V bit is set in `misa`; otherwise the claim is Zve64d only. Xsvm must be off on any V-claiming image.

11.2.7 Key constants (funct6 / width / mop)

Operation	Constant	Value
`vadd`	funct6	000000
`vsub`	funct6	000010
`vand/vor/vxor`	funct6	001001/10/11
`vsll/vsrl/vsra`	funct6	100101/101000/101001
`vsadd`	funct6	100001
`vwmul`	funct6	111011
`vfmacc`	funct6	101100
width 8/16/32/64	W field	000/101/110/111
mop unit/stride/idx	P field	00/10/01 or 11

Any mnemonic required by a wider profile but absent from a narrower one is an illegal-instruction trap on the narrower core. The bit layouts above are identical on every machine; only the legal SEW, the legal mop values, and the legal funct6 set change.

11.3 Parameters

The vector unit is one Bluespec package. The profile is an elaboration parameter, not a source fork.

```
typedef struct {
  Integer elen;
  Integer vlen;
  Integer lanes;
  Bool fp32;
  Bool fp64;
  Bool indexed;
  Bool xsvm;
  Bool claimV;
} VecParams deriving (Bits, Eq);
```

Legal combinations are exactly the four rows of the profile card. A function `paramsOk` returns false for every other combination, and the top-level module refuses to elaborate if it returns false. In particular `fp32` is false when `elen < 32`, and `claimV` is false when `elen < 64` or `vlen < 128` or `fp64` is false.

11.4 Interfaces

```
interface VectorIssue;
  method Action req(Parcel p, Regs rs);
  method Bool ready;
endinterface

interface Lane;
  method Action enq(Elem a, Elem b, Opcode op, Bool mask);
  method ActionValue#(Elem) deq;
endinterface
```

The issue method is called from the hart's execute stage for OP-V parcels. `ready` is false while an in-order vector instruction occupies the unit. Loads use a third interface, `VecMem`, whose `beat` method is granted by the same arbiter as the video master on BX-8 and BX-16.

Scalar writeback of `vsetvl`'s destination register is a method on the hart, not a side path. The vector unit returns the granted `vl` in the result structure of `req`.

11.5 The configuration rule

One rule handles `vsetvl`, `vsetvli`, and, where the profile includes it, `vsetivli`. It computes the requested SEW and LMUL, compares them with `paramsOk` for this instruction, and either updates `vtype` and `vl` or sets `vill`. The rule is atomic, so a configuration cannot be observed half-written by a following vector instruction in the same schedule. Give the rule a higher urgency than the arithmetic rule so that a `vsetvl` decoded in cycle n is visible to an instruction decoded in cycle $n + 1$.

11.6 Arithmetic rules

Integer rules are always elaborated. Their bodies case on SEW and produce an illegal-instruction side effect when the SEW is outside the profile. That side effect is a second line of defence behind `vill`. The first line is the configuration rule. The second line catches an instruction that ignores `vtype`.

Floating-point rules are wrapped in `if (p.fp32)` blocks so that a BX-8 elaboration does not instantiate DSP blocks for a single-precision multiplier. The same pattern gates FP64, indexed address generation, and the Xsvm decoder.

11.7 What SVM-8 still contains

The SVM-8 elaboration still contains the decode for opcodes it will trap: the illegal path is a rule, and the rule is source text. That is intentional. A later profile turns the same decode into a data path by changing parameters, not by pasting a second decoder. Keep the trap path in the source even when you are sure the profile will never enable the opcode. The tests of Chapter 13 include an illegal SEW on every profile, and they need something in the design that raises it.

Chapter 12

Bring-Up, from `vsetvl` to a Running Kernel

12.1 Order

The order is the same on every profile. Do not start with the widest machine on the theory that it contains the others. Start with BX-8, where a mistake is a byte lane, and carry the passing tests upward.

1. Scalar RV32I: loads, stores, branches, and a UART or a memory-mapped trace port.
2. `vsetvl` of a legal SEW, reading back `vl` and `vtype`.
3. `vsetvl` of an illegal SEW, observing `vill` and an illegal-instruction trap on the next vector opcode.
4. One vector add of `vl` elements, checked against a scalar loop.
5. Unit-stride store and load of those elements, including `vl` shorter than a register.
6. A masked add, undisturbed tail, checked byte by byte or halfword by halfword.
7. Only then the extra instructions of the profile: widening on BX-16, FP32 on BX-32, FP64 on BX-64.

12.2 The first vector add

Use SEW equal to ELEN and LMUL equal to 1. Fill two registers by stores to memory and a unit-stride load, or, before loads work, by a debug write to the file that is not part of the architecture and is compiled out of the board image. Prefer the load as soon as it exists, so that the debug write does not become a second programming model.

Compare the destination against the sum computed on the host in the testbench. Do this for `vl` from 0 through VLMAX. `vl` of 0 must retire no elements and must not raise a trap. It is the usual end condition of a strip-mined loop, and implementations get it wrong when the lane enable is a decrement that underflows.

12.3 Bus contention on the compatible machines

On BX-8, repeat the fill while the video processor is fetching a scanline. The architectural result must match the quiet-bus result. The cycle count may be longer. On BX-16, repeat a sixteen-bit fill while the audio master takes its reserved slots. A test that passes only with the other masters held in reset has not shown that the machine can replace an MSX or an Amiga. It has shown that the arithmetic works.

12.4 When to leave the simulator

Leave the simulator after the fill test and the illegal-SEW test pass on the profile you intend to synthesise. Synthesis of a vector unit that fails `vl`=0 is a long way to learn a one-line bug. The budget chapter assumes the tests are already green in simulation.

Chapter 13

Tests That Travel Up the Ladder

13.1 One suite, four expectations

The architectural tests are a single directory. Each test names the narrowest profile that must run it. A wider profile runs it too. A narrower profile does not assemble it, or assembles it and expects an illegal-instruction trap, and the expected trap is itself a test.

Test	First profile	Also required on
`fill8`	SVM-8	all wider
`mask8`	SVM-8	all wider
`sew16`	SVM-16	BX-32, BX-64
`vwmul16`	SVM-16	BX-32, BX-64
`sew32`	Zve32f	BX-64
`fma32`	Zve32f	BX-64
`sew64`	Zve64d or V	BX-64 only
`fma64`	Zve64d or V	BX-64 only
`badsew`	each profile	trap on that profile

13.2 The fill

```
// n bytes at dst become the byte fill
// assembled once, linked into every image
void fill8(uint8_t *dst, uint8_t fill, size_t n);
```

The reference is a scalar loop in the testbench, not in the hart. The hart's result is read back through the bus with the vector unit idle, so that a buggy load unit cannot hide a buggy store unit by reading its own write buffer. Compare all `n` bytes, including when `n` is one more than a multiple of VLMAX. That last iteration is the one that sets `vl` to a remainder.

13.3 Illegal SEW

On each image, execute `vsetvli` with a SEW the profile forbids: 16 on SVM-8, 32 on SVM-16, 64 on Zve32f. Then execute `vadd.vv`. The add must trap. `vl` must be unchanged from the

previous legal configuration, or zero if there was none, according to the rule you wrote down for `vill`. The test checks the CSR, not only the trap, because a trap with a silently updated `vl` will break the next legal loop.

13.4 Xsvm tests are separate

Tests that emit the custom matrix or cross-product opcode live in another directory and run only on elaborations with `xsvm` true. They are not part of the travelling suite. A clean BX-16 image must fail those tests by trapping, and the travelling suite must still pass.

13.5 Random parcels

After the directed tests, a small generator that emits legal OP-V for the profile, runs the parcel on the hart and on a scalar reference model, and compares the destination register, will find the lane-enable bugs that VLMAX and `vl`=1 do not catch. Bound it. A few thousand parcels per profile are enough for a teaching design. Record the seed.

Chapter 14

What Fits on the First FPGA

14.1 The file, in bytes

Thirty-two registers times VLEN/8 is the architectural file.

Machine	VLEN	File	First RAM style
BX-8	32	128 B	distributed
BX-16	128	512 B	distributed or one block
BX-32	128	512 B	block, replicated if ports demand
BX-64	256	1 KiB	block

These numbers are the vector file alone. The video memory of an MSX-compatible image, and the chip memory of an Amiga-compatible image, are larger and are the memories you should budget first. The vector file is not the part that will surprise the map.

14.2 DSP blocks

An eight-bit add does not need a DSP block. A sixteen-bit multiply does, or it needs a multi-cycle shift-add, which is acceptable on the first BX-16 image if the cycle count is published. A single-precision fused multiply-add wants the DSP column the FPGA actually has; read the primitive and match the lane to it instead of inferring a behavioural `*` and hoping. A double-precision unit is several primitives or a sequenced mantissa multiplier. That is why the recommended first BX-64 image has two lanes, not eight.

14.3 The first images

The first BX-8 image contains the peripheral set required by the compatibility checklist, one integer lane of SVM-8, and a UART. It does not contain Xsvm, a second hart, or a floating-point unit.

The first BX-16 image contains the sixteen-bit bus, the blitter-class master, video and audio masters, and four lanes of SVM-16 without Xsvm. Geometry, if you want it in hardware, is a second image.

The first BX-32 image contains one hart, VLEN 128, four integer lanes, and one FP32 pipeline if the device is small. The first BX-64 image contains one hart, VLEN 128, Zve64d rather than a rushed V, and two lanes.

14.4 Clocks

Use one clock for the hart, the vector unit, and the peripheral bus on BX-8 and BX-16. A separate pixel clock is allowed for the display output, with a dual-clock FIFO you can explain. Do not clock the vector lanes faster than the bus on the compatible machines. The bus is the limit, and a faster lane only builds a queue in front of the arbiter.

On BX-32 and BX-64 a faster core clock than the memory clock is reasonable. Cross it with an asynchronous or mesochronous FIFO whose depth is in the profile card, and test a vector load that drains it.

14.5 A finished first board

A first board is finished when three things are true. The travelling tests for that profile pass on the hardware, read back through the bus. The picture and the sound meet the compatibility checklist for BX-8 or BX-16, or the memory test meets the map you published for BX-32 or BX-64. The illegal-SEW test traps on the hardware, not only in simulation. Cycle counts may differ from the simulator if the arbiter is busier. Architectural results may not.

Profile Card

	SVM-8	SVM-16	Zve32f	V / Zve64d
Machine	BX-8	BX-16	BX-32	BX-64
Replaces	MSX	Amiga	—	—
XLEN	32	32	32	64
Datapath	8	16	32	64
ELEN	8	16	32	64
VLEN	32	128	≥ 128	≥ 128
EEW	8	8, 16	8–32	8–64
FP	—	—	FP32	FP32, FP64
`misa`.V	no	no	per spec	only if V
Xsvm	no	optional	optional	off on V image
Tail policy	und.	und.	und.	und.

Discovery on SVM-8 and SVM-16 is the custom CSR you name here, not the V bit.

Custom CSR address: _____
`vsetivli` on SVM-8: yes / no
Indexed load from: BX-32
`vstart` precise from: BX-64 with a page table
Lanes in the first image: 1, 4, 4, 2

Checklist against RVV 1.0

For each instruction you implement, the encoding matches the specification, the register numbers are the specification's, and an illegal SEW traps. This list is the minimum the travelling tests exercise. It is not the whole of V.

- Configuration: `vsetvl`, `vsetvli`, and `vsetivli` where the card includes it.
- `vill` on an illegal SEW or LMUL, without a surprising update of `vl`.
- Unit-stride load and store at every legal EEW.
- Stride load and store from SVM-16 upward.
- Indexed load and store from BX-32 upward.
- Integer add, subtract, logic, shift, vector-vector and vector-scalar.
- Compare and mask logicals where the card includes them.
- Saturating add where the card includes it, with `vxsat`.
- Widening multiply and narrowing shift on SVM-16, BX-32, and BX-64.
- Integer reduction and slide-by-one from SVM-16 upward.
- FP32 fused multiply-add and conversions on BX-32 and BX-64.
- FP64 fused multiply-add on BX-64.
- Undisturbed tail and inactive mask elements.
- `vl` = 0 retires nothing and does not trap.
- No claim of Zve32x on BX-8 or BX-16.
- No V bit unless every instruction required by V is present.

MSX and Amiga Compatibility Checklist

BX-8, in place of an MSX

- Eight-bit data bus shared by the hart's data port, the video display processor, and the sound generator.
- Slot and cartridge signals documented against the MSX reference you follow.
- Video modes you claim: register map, timing, and a captured frame compared to that reference.
- Sound: tone, noise, and envelope registers, and a captured waveform.
- Keyboard and joystick ports in the documented map.
- Video refresh can take the bus during a vector load; the picture remains stable; the load's bytes match a quiet-bus run.
- The hart is RV32I with SVM-8. Guest binaries of the original processor run only through software you have written for that purpose, and that software is named on the card.

BX-16, in place of an Amiga

- Sixteen-bit shared bus with hart, blitter-class master, video, and audio.
- A reserved-slot or priority rule for audio, with a measured worst-case wait.
- Display fetch can be driven from a programmed list.
- Blitter-class master, if present, is a peripheral, not a second instruction set.
- A vector fill of halfwords matches a scalar reference while audio slots are taken.
- The hart is RV32I with SVM-16. Xsvm is off on the compatibility image.

BX-32 and BX-64

These images are compatible with the RISC-V strings on their profile cards, and with the travelling tests. They are not MSX or Amiga replacements. Their checklist is the RVV checklist plus the memory map you publish.

Exercises

1. Elaborate SVM-8 with one lane and with four lanes. Show that `fill8` matches and that the cycle counts differ by about four on a long quiet-bus run.

2. Add a video master that steals every other bus cycle. Show that `fill8` still matches.

3. Implement `vill` for SEW=16 on SVM-8. Write the test that reads both the trap and the CSR.

4. Add widening multiply on SVM-16. Produce a thirty-two-bit product of two sixteen-bit lanes and a narrowing shift back to sixteen bits with saturation. Compare with a scalar reference.

5. Implement Xsvm's cross product on a second BX-16 image. Show that the compatibility image traps on that opcode and still passes the travelling suite.

6. Bring FP32 up on BX-32 for `vfmacc.vv` at SEW=32. Publish lanes of integer work and lanes of floating-point work separately if they differ.

7. Take the SVM-16 `vwmul` test binary and run it, unchanged, on BX-32 and on BX-64.

8. Implement `vstart` for a fault in the middle of a unit-stride load on BX-64. Show a restart that retires each element once.

9. Draw the scoreboard for three vector register numbers and a pending load. List the pairs that may overlap and the pairs that must not.

10. Write the `paramsOk` function and a table of ten illegal parameter records that must not elaborate, including FP32 with ELEN 16 and `claimV` with VLEN 64.

Glossary

BX-8
MSX-compatible FPGA machine. RV32I, eight-bit data path, SVM-8.

BX-16
Amiga-compatible FPGA machine. RV32I, sixteen-bit data path, SVM-16.

BX-32
RV32 machine with Zve32f.

BX-64
RV64 machine with Zve64d or V.

Datapath width
Width of the integer ALU and the data bus. Not XLEN.

ELEN
Widest vector element the implementation supports.

Profile
A named, unratified subset of RVV behaviour defined by this book: SVM-8 or SVM-16.

SVM
Scalar–vector–matrix. The scalar hart plus an RVV profile; the matrix operations are Xsvm and are optional.

VLEN
Number of bits in one of the thirty-two vector registers.

XLEN
Width of an integer register. 32 until BX-64, then 64.

Xsvm
Optional custom-0 geometry instructions. Not part of a profile. Off on a V-claiming image.

References

1. RISC-V International. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture.* Vector Extension, version 1.0, and the embedded vector subsets Zve32x, Zve32f, Zve64x, Zve64f, and Zve64d.

2. RISC-V International. *Volume II: Privileged Architecture.* Machine ISA register, traps, and the illegal-instruction exception.

3. M. Cavalcante, F. Schuiki, F. Zaruba, M. Schaffner, L. Benini. Ara: A 1 GHz+ scalable RISC-V vector processor with multi-precision floating-point support in 22 nm FD-SOI. *IEEE Transactions on VLSI Systems*, 2020.

4. M. Perotti, M. Cavalcante, and others. Ara2: an RVV 1.0 vector processor, and the multi-core study built on it. *IEEE Transactions on Computers*, 2024.

5. M. Perotti, S. Riedel, M. Cavalcante, L. Benini. Spatz: clustering compact RISC-V vector units. arXiv:2309.10137.

6. Vicuna and Vicuna 2. Open Zve32x, and later Zve32f, coprocessors using the CORE-V extension interface. `vproc` project.

7. MSX technical handbooks and datapacks, for the peripheral contract of BX-8.

8. Amiga hardware references for the chipset bus, blitter, and display fetch, for the peripheral contract of BX-16.

9. Bluespec SystemVerilog language reference, for the rule semantics assumed in Chapter 14.

The BX notes `btron-bx8`, `btron-bx16`, `btron-bx32`, `btron-bx64`, and `btron-bx-series` record the same ladder. Where an older paragraph in those notes speaks of a private sixteen-bit vector encoding, this book is the later contract: standard OP-V, thirty-two vector registers, and the profile card.